MACHINE LEARNING & GENAI INTERVIEW PREPARATION

Part 0: NLP Fundamentals: Vector Semantics, Sequence Modeling, and Classic Retrieval

Comprehensive Classical Foundations & Mathematical Study Guide

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Core Modules: Preprocessing pipeline transitions, Subword Tokenizers (BPE/WordPiece/Unigram), TF-IDF matrix derivations, Feature Hashing, Classic Retrieval (Boolean/BM25), Statistical LMs, Word2Vec/GloVe/FastText embedding geometry, RNN vanishing gradient proofs, LSTM constant error carousel, Self-Attention scaling derivation ($1/\sqrt{d_k}$), evaluation loops, and 40 Q&As

Includes: BPE merge simulator, TF-IDF hand-calculations, Katz backoff probability derivations, scaled softmax dot-product variance proofs, and production error-correction loops.

Table of Contents

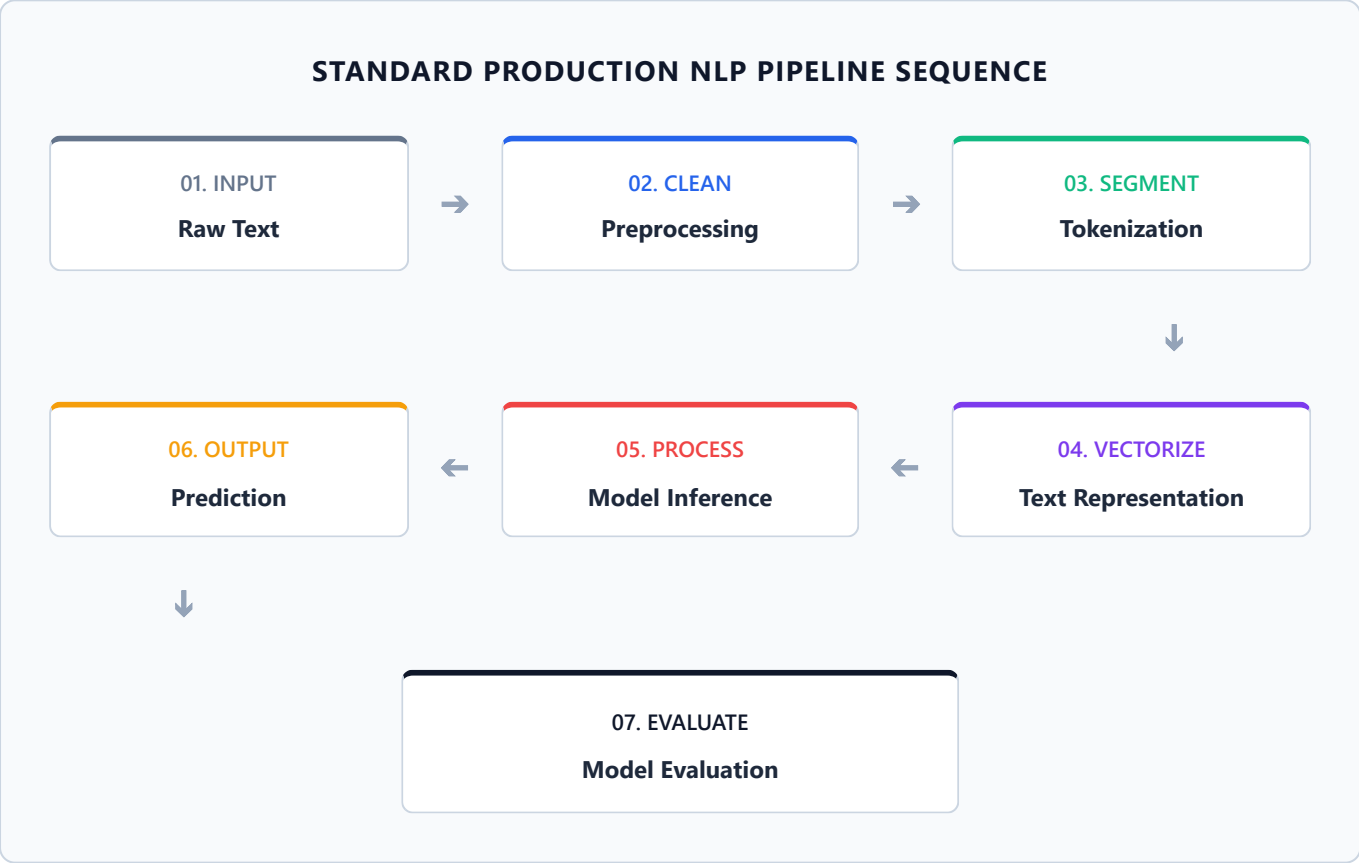
Module 01: NLP Introduction

Natural Language Processing (NLP) bridges the gap between unstructured human language and quantitative computation. This module details the standard NLP text pipeline, maps the historical paradigm shifts from rule-based filters to modern Transformers, and contrasts tokenization strategies.

---

1. The Standard NLP Pipeline

In production systems, processing raw text requires translating unstructured strings into formatted numerical matrices through a sequence of modular processing steps:



---

2. Tokenization Strategies: Character vs. Word vs. Subword

Text representation requires splitting raw characters into discrete computational units. Selecting the correct boundary limits token vocabulary size while preserving semantics:

Tokenization Strategy	Vocabulary Size	Out-of-Vocabulary (OOV) Risk	Computational Overhead	Key Bottleneck
Character-Level	Minimal (typically $\approx 100–250$ tokens covering alphabet/symbols).	Zero (0% OOV rate, every string is composed of raw characters).	High sequence length (forces models to process long arrays).	Discards semantic meaning of multi-character root words.
Word-Level	Massive (often $> 1,000,000$ tokens to cover entire dictionary).	Extremely High (unseen words like "tokenization" fail to map).	Low sequence length (one token per word).	Vocabulary explosion; unable to share semantic roots.
Subword-Level	Controlled (typically fixed at $32,000–256,000$ tokens).	Zero (unknown words are split into characters or byte tokens).	Balanced sequence length.	Requires prefix/suffix tracking character indicators (e.g. `##` or ` `).

Why Subword Tokenization Became Necessary for LLMs

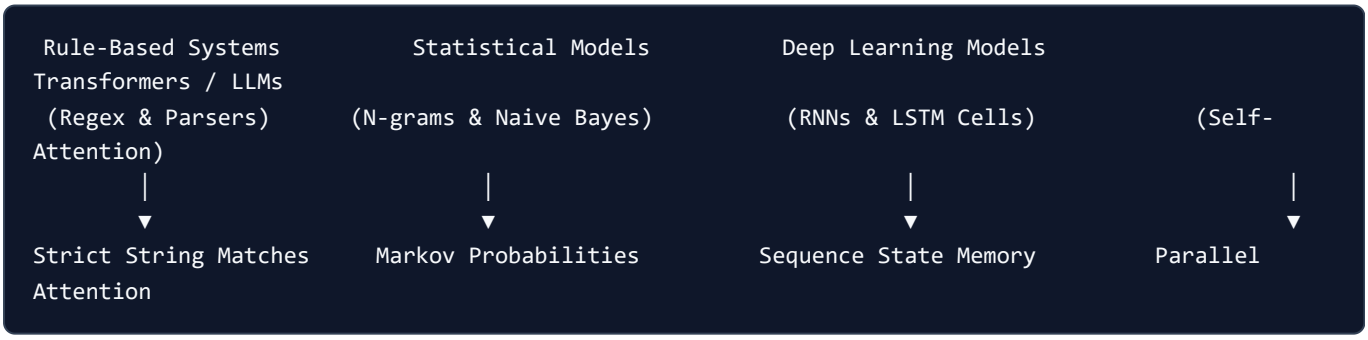
To scale models to hundreds of billions of parameters, vocabulary matrices must remain compact. Word-level tokenizers result in massive lookup matrices ($|V| \times d_{\text{model}}$), draining GPU VRAM. Character-level tokenizers increase sequence length, scaling the attention matrix cost $O(L^2)$ quadratically.

Subword tokenization bridges this gap. By splitting words into common root combinations (e.g., `"unhappy"` $\rightarrow$ `["un", "happy"]`), the vocabulary remains small while unknown words are decomposed without crashing the model.

---

3. The Evolutionary Shifts in NLP

The architecture of text systems transitioned through four distinct developmental paradigms:



1. **Rule-Based Systems**: Reliant on manual regular expressions and grammatical syntax trees. Failed to handle natural language variation or domain shifts.
2. **Statistical Models**: Modeled language probabilistically using frequency counts and Markov assumptions. Limited to short contexts due to exponential scaling of n-gram count tables.
3. **Deep Learning Sequence Models**: Introduced recurrent loops (RNNs, LSTMs) to maintain continuous hidden state vectors across variable sequence lengths. Suffered from sequential bottleneck delays during training.
4. **Transformers (Modern GenAI)**: Replaced recurrence entirely with Self-Attention query-key-value projections. Enabled massively parallel training calculations and long-range context mapping.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Establishing a standard text processing pipeline translates unstructured human language strings into dense numerical formats that machine learning decoders can optimize.
- **Why was it introduced?**
  Introduced to solve the high lexical variation, structural complexity, and unbounded dimensionality of human speech.
- **What are its limitations?**
  Stochastic processing results in text representation drift; cleaning steps can discard semantic metadata (such as negation or emojis).
- **Computational Complexity (Time & Memory)**
- **Time**: Subword tokenization using pre-compiled trees runs in $O(L)$ time, where L is the string character length.
- **Memory**: Vocabulary weight matrix footprint scales as $O(|V| \cdot d)$ where $|V|$ is the vocabulary size and d is embedding dimensionality.
- **Component Variable Denotation Legend**
- L: Character length of raw input text.
- $|V|$: Vocabulary token size.
- d: Hidden dimension size of embedding layer.
- **Production Use Cases**
- Parsing streaming customer queries for high-volume routing models.
- Subword vocabulary compression in multilingual translation pipelines.
- **Follow-up questions interviewers ask**
- *Why does word-level tokenization fail on specialized biomedical text?* (Specialized terminology features high Out-of-Vocabulary occurrences, forcing word-level models to assign the generic `<unk>` token to critical clinical terms).
- *How does the subword vocabulary limit affect training cost?* (A larger vocabulary $|V|$ increases classification layer parameter count, increasing memory usage during backpropagation calculation steps).

Module 02: Text Preprocessing & Subword Tokenization

Preprocessing translates unstructured text strings into normalized token tokens. This module details classical cleaning methods and explains the step-by-step mathematical algorithms behind modern subword tokenizers (BPE, WordPiece, Unigram).

---

1. Modern Subword Tokenization Algorithms

Modern language models avoid word-level and character-level limitations by building subword vocabularies. The three primary subword algorithms differ in how they build and prune vocabularies:

1. Byte-Pair Encoding (BPE)

BPE (used in GPT-2, GPT-3, GPT-4, LLaMA, RoBERTa) builds its vocabulary from the bottom up, iteratively merging the most frequent adjacent character pairs.

Step-by-Step Algorithm:

1. **Initialize Vocabulary**: Populate vocabulary $|V|$ with all unique characters occurring in the training corpus, plus an end-of-word marker (e.g. `</w>` or `_`).
2. **Decompose Corpus**: Split all words in the training corpus into characters. For example, the corpus `"low low low lower lower newest newest"` is represented as:
   `{'l o w </w>': 3, 'l o w e r </w>': 2, 'n e w e s t </w>': 2}`
3. **Count Pairs**: Scan the corpus to count all adjacent token pairs (e.g. `(l, o)`, `(o, w)`, `(e, s)`).
4. **Merge Most Frequent**: Find the single most frequent adjacent pair. Add it to the vocabulary as a merged token.
   - Example: The pair `(e, s)` appears 2 times in `"newest"`. The pair `(l, o)` appears 5 times. Thus, merge `l` and `o` into `lo`.
   - Update the corpus: `{'lo w </w>': 3, 'lo w e r </w>': 2, 'n e w e s t </w>': 2}`
5. **Iterate**: Repeat steps 3 and 4 until the vocabulary reaches the target size $|V|$ or the maximum frequency drops below a threshold.

---

2. WordPiece

WordPiece (used in BERT, DistilBERT, MobileBERT) is a bottom-up tokenizer similar to BPE, but instead of merging by raw frequency, it merges pairs that maximize the likelihood of the corpus under a probabilistic unigram model.

Step-by-Step Algorithm:

1. **Initialize Vocabulary**: Populate vocabulary $|V|$ with all individual characters and subwords.

2. **Calculate Merge Scores**: Compute a score for every adjacent pair of tokens (A, B) using:

$$\text{Score}(A, B) = \frac{\text{Count}(A, B)}{\text{Count}(A) \times \text{Count}(B)}$$

   Intuition: This ratio measures how often A and B appear together compared to their independent occurrences. A high score means A and B are strongly coupled (e.g. `"h"` and `"ug"` in `"hug"`).

3. **Merge**: Add the pair (A, B) with the highest score to the vocabulary.
4. **Iterate**: Repeat steps 2 and 3 until the target vocabulary budget is met.

---

3. Unigram Language Modeling

Unigram (used in T5, ALBERT, SentencePiece models) operates in a top-down manner. It starts with a massive vocabulary and iteratively prunes the least useful tokens.

Step-by-Step Algorithm:

1. **Initialize Vocabulary**: Define a very large vocabulary $|V_{\text{initial}}|$ consisting of all characters and the most common substrings/words from the training corpus.
2. **Estimate Unigram Model**: Train a unigram language model to estimate probabilities $P(x)$ for all tokens $x \in V$ using Expectation-Maximization (EM). The likelihood of a word W composed of segmentations $\{x_1, \ldots, x_k\}$ is:

$$P(W) = \prod_{i=1}^{k} P(x_i)$$

3. **Compute Pruning Loss**: For each token x in the current vocabulary, calculate the decrease in corpus log-likelihood if x were removed.
4. **Prune**: Remove the bottom $p\%$ of tokens (usually 10–20%) that result in the smallest reduction in corpus likelihood.
5. **Iterate**: Repeat steps 2-4 until the vocabulary size shrinks to the target budget $|V|$.

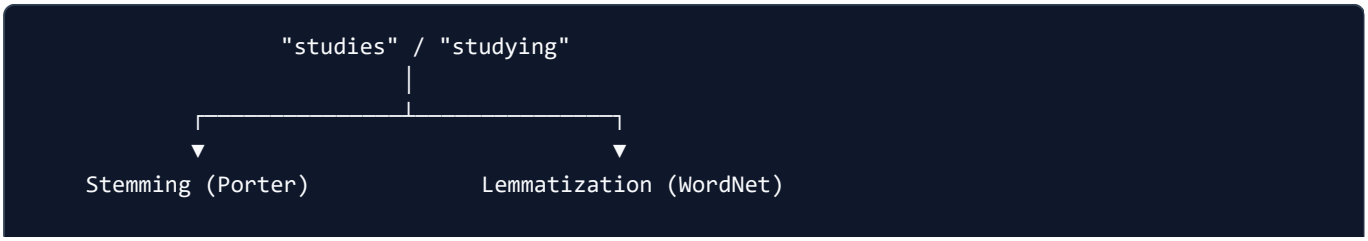
---

2. Classical Preprocessing: Normalization, Stemming, and Lemmatization

Before vectorization, classical pipelines use deterministic text normalizations:

Stemming vs. Lemmatization

Understanding the algorithmic trade-offs of these lexical reductions is a common system design question:



```
(Heuristic truncation)          (Morphological Lookup)
         |                               |
         ▼                               ▼
      "studi"                         "study"
(Fast, non-dictionary,          (Slower, dictionary-backed,
results in non-words)            preserves real root word)
```

- **Stemming (Porter Stemmer)**: A fast, rule-based heuristic that chops off word prefixes and suffixes.
- *Example*: `"studies"`, `"studying"`, `"studied"` all map to `"studi"`.
- *Failure mode*: Over-stemming (collapsing words with different meanings: `"organization"` and `"organs"` both to `"organ"`) or under-stemming (failing to merge `"alumnus"` and `"alumni"`).
- **Lemmatization (WordNet)**: Uses grammatical tags (POS tagging) and dictionary tables to resolve words to their canonical base forms (lemmas).
- *Example*: `"better"` maps to `"good"`, `"was"` maps to `"be"`.
- *Trade-off*: Slower and memory-intensive due to dictionary lookups and POS dependencies.

---

3. Regular Expressions and Unicode Normalization

Uncleaned raw strings contain encoding anomalies, boilerplate, and noise that must be filtered out:

- **Regular Expressions (Regex)**: Used to filter out URLs (`https?://\S+`), HTML tags (`<[^>]+>`), or punctuation (`[^\w\s]`).
- **Unicode Normalization**: Ensures consistent character representations. For example, the accented character `é` can be represented as:
- **NFC (Canonical Decomposition, followed by Canonical Composition)**: Single code point `\u00e9`.
- **NFD (Canonical Decomposition)**: Decomposed into base `e` (`\u0065`) and combining accent character `´` (`\u0301`).
- *Production Rule*: Always apply NFC normalization to guarantee consistent token mapping in tokenizers.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Text preprocessing transforms raw, inconsistent character streams into normalized, uniform inputs, reducing vocabulary size and vocabulary sparsity.
- **Why was it introduced?**
  Introduced to solve out-of-vocabulary (OOV) errors, normalize character variations, and compress vocabulary matrices for efficient training.
- **What are its limitations?**
  Extreme preprocessing (e.g. discarding casing and punctuation) discards semantic context (such as sentiment cues, code syntax symbols, or structural boundaries).
- **Computational Complexity (Time & Memory)**
- **BPE Encoding Time**: $O(L \cdot \log |V|)$ where L is sequence length.

- **Lemmatization Time**: $O(N \cdot D)$ where N is word count and D represents dictionary search depth.
- **Component Variable Denotation Legend**
- L: Sequence token length.
- $|V|$: Target vocabulary size.
- D: Dictionary lookup size.
- **Production Use Cases**
- Subword tokenization pipelines in multilingual LLMs.
- Normalizing raw user queries (e.g. casing and Unicode accents) before search indexing.
- **Follow-up questions interviewers ask**
- *How does BPE handle a word with unknown characters during inference?* (Characters not seen during training are mapped to character-level bytes or fallback tokens using a byte-fallback vocabulary, preventing `<unk>` errors).
- *Why should you avoid lowercase normalization for Named Entity Recognition (NER)?* (Named entities like `"Apple"` (company) depend heavily on capitalization to differentiate them from `"apple"` (fruit)).

Module 03: Text Representation & Classical Retrieval Models

Text representation maps natural language tokens into mathematical vector spaces. This module derives sparse vector formats, provides step-by-step TF-IDF calculations, details the Feature Hashing trick, and introduces Boolean and BM25 retrieval models.

---

1. Sparse Vector Formats: One-Hot, Bag of Words, and N-grams

Sparse representations construct high-dimensional vector spaces where each dimension represents a specific vocabulary word or n-gram:

- **One-Hot Encoding**: Represents each word as a binary vector of vocabulary size $|V|$, containing a single `1` at the word's index.
- *Bottleneck*: Vectors are orthogonal; $\mathbf{v}_{\text{cat}}^T \mathbf{v}_{\text{feline}} = 0$, capturing zero semantic similarity.
- **Bag of Words (BoW)**: Represents a document as a vector of word counts, ignoring syntax and word order.
- *Bottleneck*: Long documents have larger counts, biasing classification and retrieval scoring.
- **N-grams**: Extends BoW by tracking contiguous sequences of N tokens (e.g. `"not bad"` as a bigram), preserving short-range word order.
- *Bottleneck*: Dimension scales exponentially as $O(|V|^N)$, leading to extreme data sparsity.

---

2. TF-IDF Derivation and Step-by-Step Hand-Calculations

Term Frequency-Inverse Document Frequency (TF-IDF) scales word counts by penalizing words that appear frequently across the entire corpus:

Mathematical Formulas

The standard smooth TF-IDF calculation used by libraries like Scikit-Learn is defined as:

$$\text{TF}(t, d) = f_{t,d} \quad (\text{raw count of term } t \text{ in document } d)$$

$$\text{IDF}(t, D) = \log\left(\frac{1 + N}{1 + \text{DF}(t)}\right) + 1$$

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Where:
- $N = |D|$ is the total number of documents in corpus D.
- $\text{DF}(t)$ is the document frequency (number of documents containing term t).
- A final L_2 normalization is applied to output vectors to prevent length bias:

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

Step-by-Step Hand-Calculation on a Tiny Corpus

Let's calculate the TF-IDF vectors for a 2-document corpus:
- Document 1 (d_1): `"cat feline"`
- Document 2 (d_2): `"feline rug"`
- Query term set (Vocabulary): `["cat", "feline", "rug"]` ($N = 2$)

1. Calculate Document Frequency (DF) and IDF

- `"cat"` : $\text{DF} = 1 \rightarrow \text{IDF} = \log\left(\frac{1+2}{1+1}\right) + 1 = \log(1.5) + 1 \approx 0.405 + 1 = 1.405$
- `"feline"` : $\text{DF} = 2 \rightarrow \text{IDF} = \log\left(\frac{1+2}{1+2}\right) + 1 = \log(1) + 1 = 0 + 1 = 1.000$
- `"rug"` : $\text{DF} = 1 \rightarrow \text{IDF} = \log\left(\frac{1+2}{1+1}\right) + 1 \approx 1.405$

2. Compute Unnormalized TF-IDF Vectors

- **Document 1 (d_1) counts**: `{"cat": 1, "feline": 1, "rug": 0}`

$$\text{Vector}_{d_1} = [1 \times 1.405,\ 1 \times 1.0,\ 0 \times 1.405] = [1.405,\ 1.0,\ 0.0]$$

- **Document 2 (d_2) counts**: `{"cat": 0, "feline": 1, "rug": 1}`

$$\text{Vector}_{d_2} = [0 \times 1.405,\ 1 \times 1.0,\ 1 \times 1.405] = [0.0,\ 1.0,\ 1.405]$$

3. Apply L_2 Normalization

- Norm for d_1: $\|\text{Vector}_{d_1}\|_2 = \sqrt{1.405^2 + 1.0^2} = \sqrt{1.974 + 1.0} = \sqrt{2.974} \approx 1.725$

$$\mathbf{v}_{d_1} = \left[\frac{1.405}{1.725},\ \frac{1.0}{1.725},\ 0.0\right] \approx [0.814,\ 0.580,\ 0.0]$$

- Norm for d_2: $\|\text{Vector}_{d_2}\|_2 \approx 1.725$

$$\mathbf{v}_{d_2} \approx [0.0,\ 0.580,\ 0.814]$$

---

3. Cosine Similarity Mechanics

Cosine similarity measures the angular orientation between two normalized vectors, ignoring magnitude differences:

$$\text{CosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}$$

For our normalized vectors:

$$\mathbf{v}_{d_1} \cdot \mathbf{v}_{d_2} = (0.814 \times 0.0) + (0.580 \times 0.580) + (0.0 \times 0.814) = 0.3364$$

Since the vectors are pre-normalized, the cosine similarity is simply the dot product: ≈ 0.336 (indicating partial overlap due to the shared token `"feline"`).

---

4. Feature Hashing (The Hashing Trick)

To avoid storing a large vocabulary dictionary in memory, production pipelines use Feature Hashing:

- **Mechanics**: Maps raw words to a fixed array index size B using a hash function:

$$\text{Index} = h(w) \pmod B$$

- **Sign Hash**: A second independent hash function $\xi(w) \in \{-1, +1\}$ scales token values to ensure expected collision errors average to zero:

$$x_i = \sum_{w:h(w)\equiv i} \xi(w) \cdot \text{Count}(w)$$

- **Advantages**: Zero vocabulary lookup table storage footprint; constant $O(1)$ index lookup speed.
- **Collision Trade-offs**: Multiple words will map to the same bucket (e.g. `"feline"` and `"rug"` could map to index 4). If B is too small, collisions introduce noise, degrading classification accuracy.

---

5. Retrieval Models: Boolean, TF-IDF, and BM25

Retrieval systems query document collections using three primary paradigms:

1. **Boolean Retrieval**: Checks documents for binary term matches using boolean operators (`AND`, `OR`, `NOT`). Offers high precision but does not rank documents.
2. **TF-IDF Retrieval**: Ranks documents by summing the TF-IDF weights of query terms.
3. **Okapi BM25 Retrieval**: A robust retrieval algorithm that scales Term Frequency non-linearly and normalizes by document length:

$$\text{Score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{\text{TF}(q_i, D) \cdot (k_1 + 1)}{\text{TF}(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

   Where:
   - k_1 (typically 1.2–2.0) controls term frequency saturation. As Term Frequency increases, the score approaches an asymptote, preventing a single term from dominating.
   - b (typically 0.75) controls document length normalization. It penalizes long documents containing query terms simply due to length.
   - $|D|/\text{avgdl}$ is the ratio of document length to the average document length across the corpus.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Translates text documents into numeric vectors to compute similarities and retrieve relevant records.
- **Why was it introduced?**
  Introduced to score term significance relative to corpus frequencies, avoiding term density bias in documents.
- **What are its limitations?**
  Sparse vectors fail to capture semantic relationships (synonyms like `"cat"` and `"feline"` remain orthogonal).
- **Computational Complexity (Time & Memory)**
- **TF-IDF Vectorization Time**: $O(N \cdot L)$ where N is document count and L is sequence length.
- **Feature Hashing Lookup Time**: $O(1)$ index lookup.
- **Component Variable Denotation Legend**
- N: Total document count.
- L: Document token sequence length.
- B: Number of feature hash buckets.
- k_1: BM25 term frequency saturation scaling parameter.
- b: BM25 document length normalization parameter.
- **Production Use Cases**
- High-speed text retrieval index systems (Elasticsearch, BM25).
- Memory-constrained text classification pipelines using feature hashing.
- **Follow-up questions interviewers ask**
- *How does BM25 prevent document length bias?* (Long documents are penalized via the $b \cdot (|D|/\text{avgdl})$ term in the denominator. This reduces the score if query terms appear in long, noisy texts).
- *Why does the sign hash function $\xi(w)MATHPLACEHOLDER15ENDMATH(Byrandomlyassigning+1or-1$ to each word, collisions cancel each other out on average, preventing systematic inflation of collision indices).

Module 04: Statistical Language Models & Smoothing

Statistical Language Models (LMs) estimate probability distributions over token sequences. This module covers sequence probability chain rules, Markov assumptions, Maximum Likelihood Estimation (MLE), smoothing algorithms, and perplexity metrics.

---

1. Sequence Probability and the Markov Assumption

A language model calculates the joint probability of a sequence of tokens $W = (w_1, w_2, \ldots, w_m)$:

The Probability Chain Rule

Using conditional probability, the exact joint probability of a text sequence is computed as:

$$P(w_1, w_2, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

The Markov Assumption

Calculating joint probabilities requires tracking long histories, which quickly becomes computationally intractable. The Markov assumption simplifies this by assuming the probability of a word depends only on the preceding k words:
- **Unigram Model** ($k = 0$): Assumes all words are independent:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i)$$

- **Bigram Model** ($k = 1$): Assumes word depends only on the immediate predecessor:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-1})$$

- **Trigram Model** ($k = 2$): Assumes dependency on the preceding two words:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-2}, w_{i-1})$$

---

2. Maximum Likelihood Estimation (MLE)

MLE calculates probabilities by counting occurrences in a training corpus:

Bigram MLE Formula

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Where $C(w_{i-1}, w_i)$ is the bigram co-occurrence count, and $C(w_{i-1})$ is the unigram count of the preceding token.

---

3. Smoothing Techniques and Good-Turing Intuition

If an n-gram never occurs in the training corpus, MLE assigns it a probability of 0. A single zero count makes the joint sequence probability collapse to 0:

$$P(W) = 0$$

Smoothing reallocates probability mass from frequent n-grams to unseen sequences.

1. Laplace (Add-One) Smoothing

Adds 1 to all counts to ensure no probability evaluates to 0:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

Where $|V|$ is vocabulary size.
Trade-off: Assigns too much probability mass to unseen words in large vocabularies.

2. Backoff (Katz Backoff)

If the higher-order n-gram count is zero, the model backs off to estimate probability using lower-order n-grams (e.g. bigram $\rightarrow$ unigram):

$$P_{\text{Katz}}(w_i \mid w_{i-1}) = \begin{cases} P^*(w_i \mid w_{i-1}) & \text{if } C(w_{i-1}, w_i) > 0 \\ \alpha(w_{i-1}) P(w_i) & \text{if } C(w_{i-1}, w_i) = 0 \end{cases}$$

Where P^* represents a discounted probability, and $\alpha(w_{i-1})$ is a normalization factor.

3. Interpolation

Linearly combines probabilities from multiple n-gram orders:

$$P_{\text{Interp}}(w_i \mid w_{i-2}, w_{i-1}) = \lambda_1 P(w_i \mid w_{i-2}, w_{i-1}) + \lambda_2 P(w_i \mid w_{i-1}) + \lambda_3 P(w_i)$$

Where $\sum \lambda_j = 1$ and $\lambda_j \geq 0$.

4. Good-Turing Smoothing (Intuition)

Good-Turing smoothing estimates the probability of unseen items based on the frequency of single-occurrence items (hapax legomena).
- *Intuition*: If you read a book and find 100 words that only occur once, it is highly likely that the next new word you encounter is similar to those single-occurrence words.
- The model adjusts counts c to a new smoothed value c^*:

$$c^* = (c + 1) \frac{N_{c+1}}{N_c}$$

Where N_c is the number of distinct n-grams that occur exactly c times in the corpus.

---

4. Perplexity (PPL) Derivation

Perplexity evaluates language model performance on a test sequence.

Mathematical Derivation

Perplexity is defined as the inverse probability of the test set, normalized by sequence length m:

$$\text{PPL}(W) = P(w_1, w_2, \ldots, w_m)^{-\frac{1}{m}} = \sqrt[m]{\frac{1}{\prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1})}}$$

By converting to log space, perplexity can be computed using cross-entropy loss $H(W)$:

$$\log(\text{PPL}) = -\frac{1}{m} \sum_{i=1}^{m} \log P(w_i \mid w_1, \ldots, w_{i-1}) = \text{Cross-Entropy Loss}$$

$$\text{PPL} = e^{\text{Loss}}$$

Intuition: Perplexity represents the average branching factor (i.e. the number of equally likely next words the model must choose from). A lower perplexity indicates a better model.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Estimates the probability distribution over text sequences, allowing models to generate coherent text.
- **Why was it introduced?**
  Introduced to score token sequences based on natural language frequencies.
- **What are its limitations?**
- **Memory Bottleneck**: Parameter count scales exponentially as $O(|V|^N)$ as the n-gram context order N increases.
- **Zero-Probability Defect**: Fails to generate probabilities for unseen sequences without smoothing.
- **Computational Complexity (Time & Memory)**
- **Inference Time**: $O(1)$ constant time lookup if using precomputed n-gram tables.
- **Storage Memory**: $O(|V|^N)$ space.
- **Component Variable Denotation Legend**
- m: Token count of the evaluation sequence.
- $|V|$: Vocabulary token size.
- N_c: Number of unique n-grams appearing exactly c times.
- λ_i: Interpolation coefficients.
- **Production Use Cases**
- Text auto-complete query routing systems.
- Basic speech recognition transcript decoding.

- **Follow-up questions interviewers ask**
- *Why does Perplexity decrease as you increase N in an N-gram model?* (Higher-order models capture more local context, reducing uncertainty at each step, which lowers cross-entropy loss and perplexity).
- *How do you choose lambda interpolation parameters?* (By optimizing the coefficients using expectation-maximization or grid search on a held-out validation dataset).

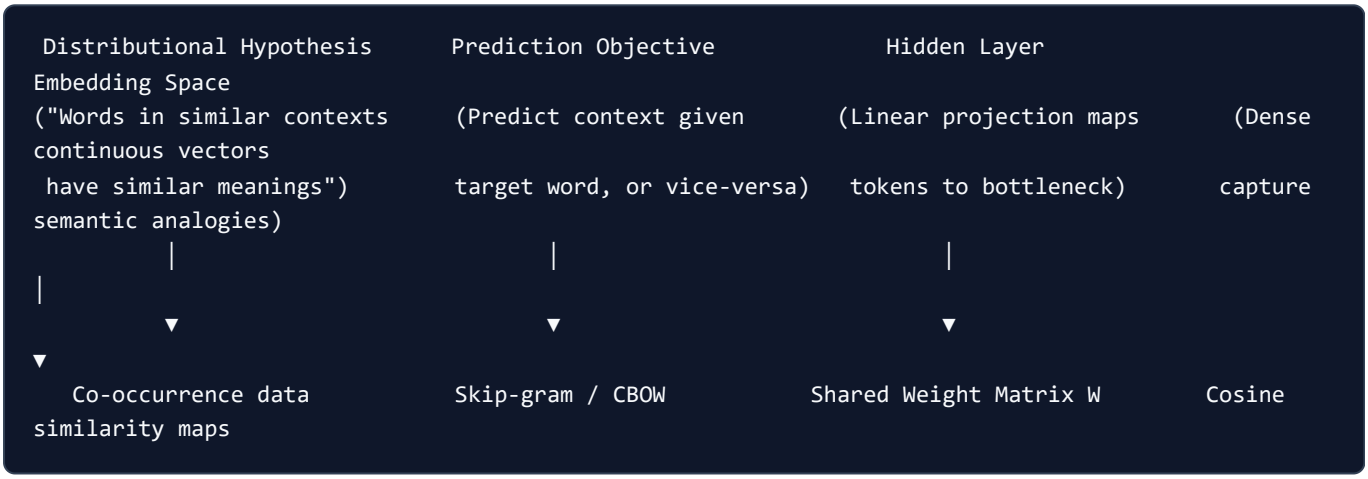
Module 05: Word Embeddings & Semantic Spaces

Word embeddings project discrete vocabulary tokens into low-dimensional, dense continuous vector spaces. This module details embedding projection theory, derives the Word2Vec, GloVe, and FastText algorithms, and explains Out-of-Vocabulary (OOV) handling.

---

1. Embedding Space Projection Theory

Word embeddings translate semantic similarity into spatial proximity (e.g. cosine distance). The learning pipeline follows four main steps:



- **Distributional Hypothesis**: Words that occur in similar contexts share semantic meaning.
- **Prediction Objective**: Rather than counting co-occurrences directly, models set up a prediction task (e.g. predicting a word from its neighbors).
- **Hidden Layer**: To solve this task, the model maps inputs through a low-dimensional bottleneck layer.
- **Embedding Space**: The weights of this bottleneck layer form the embedding matrix $\mathbf{W} \in \mathbb{R}^{|V| \times d}$, capturing semantic properties like analogies (e.g., $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$).

---

2. Word2Vec: CBOW vs. Skip-gram

Word2Vec uses local context window predictions to train embedding vectors:

Continuous Bag-of-Words (CBOW)

CBOW predicts a target word w_t given context words within a window size k:

$$\mathbf{h} = \frac{1}{2k} \sum_{-k \le j \le k, j \neq 0} \mathbf{v}_{w_{t+j}}$$

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}'\mathbf{h})$$

Where $\mathbf{v}$ are input embeddings, $\mathbf{W}'$ is the output projection matrix, and $\mathbf{h}$ is the average context vector.

Skip-gram

Skip-gram predicts context words $w_{t-k}, \ldots, w_{t+k}$ given target word w_t:

$$P(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{v}'_{w_{t+j}} \cdot \mathbf{v}_{w_t})}{\sum_{w \in V} \exp(\mathbf{v}'_w \cdot \mathbf{v}_{w_t})}$$

---

3. Negative Sampling Optimization (SGNS)

Calculating the denominator of the Softmax function over a large vocabulary $|V|$ is computationally expensive. Negative Sampling converts this multi-class classification into binary logistic regression:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}'_{w_O} \cdot \mathbf{v}_{w_I}) - \sum_{i=1}^{K} \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-\mathbf{v}'_{w_i} \cdot \mathbf{v}_{w_I})]$$

Where:
- w_O is the true output context word.
- w_I is the input target word.
- w_i are K negative sample words drawn from a noise distribution $P_n(w)$.
- $P_n(w)$ is the unigram count raised to the $3/4$ power to boost the probability of sampling rare words:

$$P_n(w) = \frac{U(w)^{3/4}}{\sum_{w'} U(w')^{3/4}}$$

---

4. GloVe (Global Vectors) and FastText

Alternative algorithms address Word2Vec's limitations:

GloVe: Global Co-occurrence Factorization

While Word2Vec scans local context windows, GloVe factorizes global co-occurrence matrices:

$$\mathcal{J} = \sum_{i,j=1}^{|V|} f(X_{i,j})(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{i,j})^2$$

Where:
- $X_{i,j}$ is the co-occurrence count of words i and j.
- $f(X_{i,j}) = \min\left(1, \left(\frac{X_{i,j}}{x_{\max}}\right)^{\alpha}\right)$ is a weighting function to prevent frequent co-occurrences from dominating.

FastText: Subword N-grams for OOV Handling

Word2Vec and GloVe cannot construct vectors for words unseen during training (Out-of-Vocabulary, OOV). FastText solves this by representing words as a bag of character n-grams.
- *Example*: For word `"where"` with $n = 3$, character n-grams are: `<wh` , `whe` , `her` , `ere` , `re>` (including boundary

markers `<` and `>`).
- The word vector $\mathbf{v}_w$ is the sum of its character n-gram embeddings:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

- *OOV Resolution*: When an unseen token is encountered during inference, FastText generates its vector by summing the embeddings of its constituent character n-grams.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Maps discrete vocabulary words to dense, low-dimensional vector representations that capture semantic similarity.
- **Why was it introduced?**
  Introduced to solve the high dimensionality and orthogonal constraints of sparse representations (One-Hot).
- **What are its limitations?**
  Static embeddings assign a single vector to each word, failing to handle polysemy (e.g. `"bank"` in `"river bank"` vs. `"money bank"`).
- **Computational Complexity (Time & Memory)**
- **Inference Lookup Time**: $O(1)$ constant time lookup.
- **Training Time**: Skip-gram with negative sampling scales as $O(K \cdot N \cdot C)$ where C is corpus size.
- **Component Variable Denotation Legend**
- $|V|$: Vocabulary size.
- d: Vector dimension size (typically 100–300).
- K: Number of negative samples.
- C: Corpus token count.
- **Production Use Cases**
- High-speed semantic similarity searches.
- Initializing embedding layers in recurrent or classification neural networks.
- **Follow-up questions interviewers ask**
- *Why does CBOW train faster than Skip-gram?* (CBOW averages context embeddings into a single vector to predict one target, while Skip-gram runs multiple predictions per target word).
- *Why is the noise distribution raised to the 3/4 power in Negative Sampling?* (It increases the relative sampling probability of rare words, ensuring the model updates rare word vectors frequently).

Module 06: Sequence Models & Recurrent Architectures

Sequence models process variable-length inputs by maintaining sequential state representations. This module details recurrent architectures, contains mathematical proofs of vanishing gradients, details LSTM cell gating, and covers sequence-to-sequence translation models.

---

1. RNNs and the Mathematical Proof of Vanishing Gradients

A standard Recurrent Neural Network (RNN) processes tokens sequentially, updating a hidden state vector h_t at each step:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Mathematical Proof of Vanishing Gradients

Consider a loss function L_T calculated at time step T. We calculate the gradient of the loss with respect to the recurrent weight matrix W_{hh}:

$$\frac{\partial L_T}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

The key term that determines gradient stability is the Jacobian chain product:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

Let's compute the individual Jacobian $\frac{\partial h_k}{\partial h_{k-1}}$:

$$\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(1 - h_k^2)W_{hh}^T$$

Where $\text{diag}(1 - h_k^2)$ is the diagonal matrix of the derivative of the activation function $\tanh$.
Substituting this back into the product:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \text{diag}(1 - h_k^2)W_{hh}^T$$

Taking the norm of this product:

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{k=t+1}^{T} \left\| \text{diag}(1 - h_k^2) \right\| \left\| W_{hh}^T \right\|$$

Since $\tanh'(x) = 1 - \tanh^2(x) \in (0, 1]$, the diagonal matrix norm is less than or equal to 1.
- **Vanishing Gradient**: If the largest eigenvalue (spectral radius) of the weight matrix satisfies $\rho(W_{hh}) < 1$, the norm

of the product decays exponentially as the time gap $T - t$ increases:

$$\lim_{T-t \to \infty} \frac{\partial h_T}{\partial h_t} = 0$$

This prevents gradient updates from propagating back to the earliest steps, making the model blind to long-range dependencies.
- **Exploding Gradient**: If $\rho(W_{hh}) > 1$, the product can grow exponentially, causing weight divergence during training.

---

2. LSTM Gating Mechanics and the Constant Error Carousel

The Long Short-Term Memory (LSTM) network introduces a dedicated cell state vector C_t to act as a linear conveyor belt for gradient flow.

LSTM Gating Equations

At step t, the LSTM updates its state vectors using four gating mechanisms:

$$\text{Forget Gate: } f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$\text{Input Gate: } i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\text{Candidate Cell State: } \tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$\text{Cell State Update: } C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$\text{Output Gate: } o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\text{Hidden State: } h_t = o_t \odot \tanh(C_t)$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ projects gate activations to $(0, 1)$, and $\odot$ is the element-wise Hadamard product.

Proof of the Constant Error Carousel (CEC)

To see how LSTM prevents vanishing gradients, we calculate the derivative of the cell state C_t with respect to the previous state C_{t-1}:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t + \text{terms containing } \frac{\partial f_t}{\partial C_{t-1}}$$

If we set the forget gate to $f_t = 1$ (indicating the model should retain historical cell information), the derivative simplifies to:

$$\frac{\partial C_t}{\partial C_{t-1}} \approx 1$$

Consequently, the error gradient can propagate back through time indefinitely without exponential decay:

$$\frac{\partial C_T}{\partial C_t} = \prod_{k=t+1}^{T} f_k \approx 1$$

This linear update path is called the **Constant Error Carousel**, allowing LSTMs to retain information across long contexts.

---

3. GRU Variations

The Gated Recurrent Unit (GRU) simplifies the LSTM by merging the cell state and hidden state, and combining the input and forget gates:

$$\text{Update Gate: } z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

$$\text{Reset Gate: } r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

$$\text{Candidate State: } \tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$\text{Hidden State: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Trade-off: GRUs have fewer parameters than LSTMs, leading to faster training times, but can exhibit lower capacity on long sequences.

---

4. Bidirectional Recurrent Models

Bidirectional sequence models process input sequences in both directions, capturing future and past context:

$$\vec{h}_t = \text{LSTM}_{\text{forward}}(x_t, \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1})$$

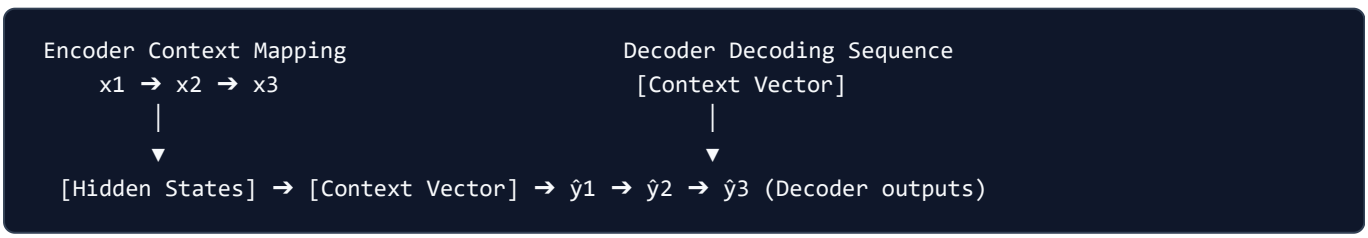
$$\text{Combined Hidden State: } h_t = [\vec{h}_t \; ; \; \overleftarrow{h}_t]$$

This combined representation captures context from both sides of a word, forming the architectural foundation for encoders like BERT.

---

5. Sequence-to-Sequence (Seq2Seq) and Decoding Strategies

Seq2Seq models use an Encoder network to compress a variable-length source sequence into a single context vector, which a Decoder network then unpacks into a target sequence:



Decoder Strategies

- **Teacher Forcing**: During training, the decoder receives the ground-truth target tokens as input instead of its own previous predictions, stabilizing early training.
- **Greedy Search**: The model outputs the most likely token at each step:
  $$y_t = \arg\max P(y \mid y_{ *Limitation*: Cannot backtrack; an early error propagates through the rest of the generation.
- **Beam Search**: Keeps a running set of the B most likely hypothesis sequences (beams). At each step, it expands all beams and retains the top B paths with the highest cumulative log probability:
  $$\text{Score}(y_{1..t}) = \sum_{i=1}^t \log P(y_i \mid y_{

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Processes variable-length sequence data while retaining historical information across time steps.
- **Why was it introduced?**
  Introduced because standard feedforward networks require fixed-sized inputs, making them unable to process sequences of arbitrary length.
- **What are its limitations?**
- **Sequential Bottleneck**: Processing token t requires computing states up to step $t-1$, preventing parallel execution.
- **Memory Footprint**: BPTT requires storing intermediate hidden states for all steps, leading to high VRAM footprint.
- **Computational Complexity (Time & Memory)**
- **Sequence processing Time**: $O(L \cdot d^2)$ where L is sequence length and d is hidden dimension size.
- **Backpropagation Memory**: $O(L \cdot d)$ per layer.
- **Component Variable Denotation Legend**
- L: Sequence token length.
- d: Recurrent hidden state dimension.
- B: Beam search branch width parameter.
- **Production Use Cases**
- Text translation pipelines.
- Named Entity Recognition sequence tagging.
- **Follow-up questions interviewers ask**
- *Why do we use log probabilities instead of raw probabilities in Beam Search?* (Multiplying small probabilities leads to numerical underflow; summing log probabilities keeps calculations numerically stable).
- *How does Gradient Clipping prevent exploding gradients in RNNs?* (If the norm of the gradient exceeds a threshold $g_{\max}$, it is scaled down: $\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{\|\mathbf{g}\|}$, preventing weight updates from causing network instability).

Module 07: Attention & Transformer Prerequisites

Attention mechanisms resolved the architectural limits of recurrent neural networks. This module details the sequence bottleneck, contrasts Bahdanau and Luong attention, defines Query-Key-Value projections, and provides the mathematical proof of the attention scaling factor.

---

1. The Seq2Seq Encoder Bottleneck

In classical sequence-to-sequence (Seq2Seq) models, the encoder compresses the entire source sentence into a single, fixed-size context vector:

$$\mathbf{c} = \mathbf{h}_L$$

This introduces the **encoder bottleneck**: the context vector must store all information from the source sentence, regardless of sequence length. If the sequence length L is long (e.g. > 30 tokens), the fixed hidden state cannot store the entire context, causing translation quality to decay.

---

2. Attention Intuition: Bahdanau vs. Luong

Attention solves this bottleneck by letting the decoder view all intermediate encoder hidden states $\mathbf{h}_i$ at each decoding step. The decoder dynamically weighs the significance of each encoder hidden state:

$$\mathbf{c}_t = \sum_{i=1}^{L} \alpha_{t,i} \mathbf{h}_i$$

Where $\alpha_{t,i}$ are attention weights representing the alignment between decoder state at step t and encoder state at step i.

Bahdanau vs. Luong Attention

- **Bahdanau (Additive) Attention**:
- The alignment score is calculated using a single-layer feedforward network:

$$\text{Score}(\mathbf{s}_{t-1}, \mathbf{h}_i) = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

- Uses the previous decoder hidden state $\mathbf{s}_{t-1}$ to calculate alignment scores.
- **Luong (Multiplicative) Attention**:
- Uses simpler multiplicative dot products, which can be computed via efficient matrix multiplications:

$$\text{Score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^T \mathbf{W}_a \mathbf{h}_i$$

- Uses the current decoder hidden state $\mathbf{s}_t$ to compute alignment.

---

3. Query, Key, and Value Intuition

Attention models map query vectors against key-value pairs:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Query (Q)**: The current token search vector (representing what information the model is looking for).
- **Key (K)**: The indexing labels of all available tokens (representing what characteristics each token offers).
- **Value (V)**: The actual content vectors of the tokens (representing the information that is retrieved).

Search Analogy: When searching for a video on YouTube, your search text is the **Query**. YouTube matches this text against video metadata tags (**Keys**) and retrieves the actual video stream (**Value**) associated with the highest-scoring match.

---

4. Mathematical Proof of the Attention Scaling Factor

Scaled dot-product attention scales query-key dot products by $\frac{1}{\sqrt{d_k}}$, where d_k is key dimension size.

Mathematical Proof

Assume the query vector $\mathbf{q} \in \mathbb{R}^{d_k}$ and key vector $\mathbf{k} \in \mathbb{R}^{d_k}$ are independent random vectors whose components are independent random variables with mean 0 and variance 1:

$$\mathbb{E}[q_i] = 0, \quad \text{Var}(q_i) = 1$$

$$\mathbb{E}[k_i] = 0, \quad \text{Var}(k_i) = 1$$

The dot product is:

$$u = \mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$$

Let's find the mean of the dot product:

$$\mathbb{E}[u] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$$

Since q_i and k_i are independent, the variance of each product term $q_i k_i$ is:

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2$$

Using independence:

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - (\mathbb{E}[q_i]\mathbb{E}[k_i])^2 = (1)(1) - (0)^2 = 1$$

Because the components are independent, the variance of the sum is the sum of the variances:

$$\text{Var}(u) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k$$

Thus, the dot product $u = \mathbf{q} \cdot \mathbf{k}$ has mean 0 and variance d_k.

Softmax Saturation and Vanishing Gradients

If the key dimension d_k is large, the variance of the dot products is high. This leads to large absolute values of u, pushing the Softmax function into regions with extremely small gradients:

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

The derivative of Softmax is:

$$\frac{\partial \sigma_i}{\partial z_j} = \sigma_i(\delta_{ij} - \sigma_j)$$

If one input z_i is much larger than the others, $\sigma_i \approx 1$ and $\sigma_j \approx 0$ (for $j \neq i$). In both cases, the derivatives are near 0, causing vanishing gradients.
Dividing by the standard deviation $\sqrt{d_k}$ scales the dot products to have a variance of 1:

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{1}{d_k}\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \frac{d_k}{d_k} = 1$$

This keeps the inputs in a range where Softmax remains sensitive to weight updates, preventing vanishing gradients during training.

---

5. Transition to LLMs: Why Transformers Replaced RNNs

The shift from recurrent models to Transformers was driven by two key limitations of RNNs:

1. **No Parallel Training**: RNN hidden state updates require sequential processing ($t-1 \rightarrow t$). Transformers process all tokens in parallel by replacing recurrent steps with self-attention matrix operations.
2. **No Long-Range Path Decay**: In RNNs, information must travel through sequential steps, causing path decay. In self-attention, the path length between any two tokens is $O(1)$, resolving the vanishing gradient problem over long sequences.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Allows models to process long-range token relationships without information bottlenecks.
- **Why was it introduced?**
  Introduced to solve the encoder bottleneck in Seq2Seq models.
- **What are its limitations?**

- **Quadratic Compute Cost**: Self-attention requires computing all query-key pairs, scaling as $O(L^2)$ time and memory.
- **Computational Complexity (Time & Memory)**
- **Self-Attention Time**: $O(L^2 \cdot d)$ where L is sequence length.
- **VRAM Memory Footprint**: $O(L^2)$ matrix storage.
- **Component Variable Denotation Legend**
- L: Token sequence length.
- d_k: Key vector dimension size.
- d: Hidden state vector dimension size.
- **Production Use Cases**
- Text translation engines.
- Parallelized token sequence learning.
- **Follow-up questions interviewers ask**
- *Why is self-attention memory cost quadratic with sequence length?* (Because it computes an $L \times L$ attention matrix storing attention weights for every query-key pair).
- *How does positional encoding help attention capture order?* (Attention is permutation-invariant; it treats tokens as a bag of words. Positional encodings add positional vectors to word vectors, letting the model distinguish token positions).

Module 08: NLP Evaluation Metrics & Semantic Validation

Evaluating natural language outputs requires measuring both exact tokens and semantic meaning. This module details classification and generation metrics, derives BLEU and ROUGE scores, highlights the limitations of n-gram overlaps, and explains semantic metrics like BERTScore.

---

1. Classification Metrics: Precision, Recall, and F1

For classification tasks (e.g. sentiment classification, token classification), performance is evaluated using confusion matrix derivatives:

- **Precision (Positive Predictive Value)**: Measures the proportion of positive identifications that were actually correct:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall (Sensitivity)**: Measures the proportion of actual positives that were correctly identified:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **F1 Score**: The harmonic mean of precision and recall, balancing both metrics when dealing with class imbalances:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

---

2. Generation Metrics: BLEU, ROUGE, and METEOR

For sequence generation tasks (e.g. translation, summarization), models are evaluated against ground-truth reference texts:

Bilingual Evaluation Understudy (BLEU)

BLEU measures n-gram precision (how many generated n-grams appear in the reference text) combined with a penalty for short generations:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Where:
- p_n is the modified n-gram precision:

$$p_n = \frac{\sum_{\text{ngram}} \text{Count}_{\text{clip}}(\text{ngram})}{\sum_{\text{ngram}} \text{Count}(\text{ngram})}$$

- w_n are uniform weights (typically 0.25 for $N = 4$).
- BP is the Brevity Penalty, which penalizes generations that are shorter than the reference:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases}$$

Where c is candidate length and r is reference length.

Recall-Oriented Understudy for Gisting Evaluation (ROUGE)

ROUGE measures n-gram recall (how many reference n-grams are captured by the generated text):
- **ROUGE-N**: Measures n-gram overlap:

$$\text{ROUGE-N} = \frac{\sum_{\text{ngram}} \text{Count}_{\text{match}}(\text{ngram})}{\sum_{\text{ngram}} \text{Count}_{\text{reference}}(\text{ngram})}$$

- **ROUGE-L**: Measures the Longest Common Subsequence (LCS) between candidate and reference texts, preserving word order without requiring exact n-gram alignments.

---

3. Semantic Blind Spots of Overlap Metrics

N-gram overlap metrics (BLEU, ROUGE) measure exact match counts. Consequently, they suffer from two major **semantic blind spots**:

1. **Orthogonal Synonyms**: A generated sentence like `"A feline rested on the rug"` compared against reference `"The cat sat on the mat"` receives a BLEU score of 0 because no words overlap, despite sharing identical meanings.
2. **Grammar & Logical Inversions**: A candidate sentence `"not bad, actually great"` compared against `"not great, actually bad"` shares identical unigrams and bigrams, but has the opposite meaning. Overlap metrics assign these sentences high scores.

---

4. Semantic Evaluation: Sentence-BERT and BERTScore

To resolve these blind spots, production systems use embedding-based semantic evaluation:

Sentence-BERT (SBERT)

Maps entire sentences to dense, fixed-sized semantic vectors $\mathbf{u}$ and $\mathbf{v}$ using a Siamese network. Similarity is measured using cosine distance:

$$\text{Similarity} = \text{CosineSimilarity}(\mathbf{u}, \mathbf{v})$$

BERTScore: Contextual Token Alignments

BERTScore computes token-level semantic alignments using contextual embeddings (e.g. BERT hidden states):

```
 Candidate:   A   feline  rested   on  the  rug
              \    /       |       |   |   /
Reference:   The  cat      sat     on  the mat
             (Aligned via Cosine Similarity of contextual tokens)
```

1. **Represent Tokens**: Map words in candidate C and reference R to contextual token vectors.
2. **Compute Similarity**: Calculate a cosine similarity matrix between all candidate and reference tokens.
3. **Greedy Matching**: Match each token in candidate C to its most similar token in reference R:

$$\text{Recall} = \frac{1}{|R|} \sum_{r_i \in R} \max_{c_j \in C} \mathbf{r}_i^T \mathbf{c}_j$$

$$\text{Precision} = \frac{1}{|C|} \sum_{c_j \in C} \max_{r_i \in R} \mathbf{r}_i^T \mathbf{c}_j$$

4. **F1 Calculation**: Compute the harmonic mean of the greedy precision and recall scores. This captures semantic similarity even when the words used are different.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Quantifies the accuracy, semantic alignment, and structural quality of natural language outputs.
- **Why was it introduced?**
  Introduced to automate translation and summarization scoring, replacing expensive human evaluation loops.
- **What are its limitations?**
  BLEU/ROUGE fail to capture semantic meaning; embedding models (BERTScore) introduce computational overhead and depend on the quality of the underlying encoder.
- **Computational Complexity (Time & Memory)**
- **BLEU Evaluation Time**: $O(L_{\text{cand}} \cdot L_{\text{ref}})$ string search.
- **BERTScore Evaluation Time**: $O(L_{\text{cand}} \cdot L_{\text{ref}} \cdot d)$ plus the forward pass latency of the encoder network.
- **Component Variable Denotation Legend**
- c: Candidate token sequence length.
- r: Reference token sequence length.
- d: Contextual embedding dimension size.
- **Production Use Cases**
- Continuous Integration (CI) regression testing for translation pipelines.
- Semantic similarity evaluations in question-answering systems.

- **Follow-up questions interviewers ask**
- *Why does BLEU use a brevity penalty?* (Without a brevity penalty, a candidate model could output a single high-confidence word like `"the"` and receive a perfect precision score of 1.0).
- *How does BERTScore handle spelling variations?* (Contextual embeddings capture semantic similarities, allowing misspelled or related words to match based on their surrounding contexts).

Module 09: Production NLP & Model Maintenance

Moving NLP models from research environments to production requires managing deployment constraints and monitoring data drift. This module details ingestion pipelines, explains data and concept drift, and maps out the production debugging feedback loop.

---

1. Production Ingestion: Batch vs. Real-Time Inference

Production systems deploy models using one of two ingestion patterns:

- **Batch Inference**: Processes large collections of text offline (e.g. running classification sweeps over nightly logs).
- *Optimization*: High throughput; optimized via parallel worker nodes and large batch sizes to maximize GPU utilization.
- **Real-Time Inference**: Processes streaming user queries with strict latency limits (e.g. query auto-completion, conversational interfaces).
- *Optimization*: Low latency; optimized via model quantization, single-sample processing, and caching.

---

2. Monitoring Production Drift: Data vs. Concept Drift

Once deployed, models experience performance decay due to environmental changes. Production monitors track two distinct types of drift:

Data Drift (Covariate Shift)

The distribution of input text changes over time, but the underlying relationship between inputs and outputs remains the same:

$$P(X_{\text{production}}) \neq P(X_{\text{training}}) \quad \text{but} \quad P(Y \mid X_{\text{production}}) = P(Y \mid X_{\text{training}})$$

- *Example*: A sentiment classifier trained on formal news articles is deployed to monitor social media comments. The vocabulary features new abbreviations and emojis, but the semantic definitions of positive and negative remain unchanged.

Concept Drift

The structural relationship between inputs and target outputs changes over time, even if the input vocabulary remains identical:

$$P(Y \mid X_{\text{production}}) \neq P(Y \mid X_{\text{training}}) \quad \text{but} \quad P(X_{\text{production}}) = P(X_{\text{training}})$$

- *Example*: The term `"viral"` shifts from representing a healthcare safety warning (2019) to representing a positive marketing trend (2021). The inputs are identical, but the target sentiment classifications invert.

---

3. Production Model Compression

To fit models within latency budgets and GPU memory allocations, production pipelines apply two primary compression techniques:

1. **Quantization**: Converts model weights from float32 (32-bit) to lower-precision formats like float16 or int8.
   - *Result*: Reduces VRAM footprint by up to 75% with minimal loss in model accuracy.
2. **Pruning**: Identifies and removes weight connections that have small gradients or magnitudes, zeroing out non-essential parameters.
   - *Result*: Increases inference speed by creating sparse weight matrices that can bypass redundant calculations.

---

4. The Complete Production Debugging Feedback Loop

Production maintenance requires a continuous monitoring and updating cycle to identify and resolve model decay:

1. **Inference**: Models process incoming user tokens and record output classifications and confidence scores.
2. **Metrics Log**: Tracks performance metrics (e.g. drop in classification confidence, spike in user fallbacks) and checks for data drift.
3. **Error Analysis**: Automatically flags low-confidence or high-loss predictions. Engineers review these flagged logs, categorizing errors into issues like subword tokenization anomalies, Out-of-Vocabulary (OOV) tokens, or class imbalances.
4. **Diagnostic Actions & Model Update**: Adjust the vocabulary mapping tables, tune classification thresholds, or retrain the model on newly drifted samples. The updated model is then validated against a golden test set and re-deployed.

Interview Questions & Production Trade-offs

- **What problem does this solve?**
  Maintains model accuracy, latency limits, and resource efficiency when running in production.
- **Why was it introduced?**
  Introduced to prevent performance decay due to domain shifts and data drift in live environments.
- **What are its limitations?**
  Pruning and quantization can lead to degraded accuracy on edge cases.
- **Computational Complexity (Time & Memory)**
- **Data Drift Detection Time**: $O(N \cdot |V|)$ where N is sample size and $|V|$ is vocabulary size.
- **Memory Footprint (int8 Quantized)**: 25% of the original float32 model footprint.
- **Component Variable Denotation Legend**

- X: Input text features.
- Y: Output label targets.
- N: Audit sample window count.
- $|V|$: Vocabulary size.
- **Production Use Cases**
- Monitoring customer service classifiers for vocabulary shift.
- Compressing models to run on edge devices.
- **Follow-up questions interviewers ask**
- *How do you identify Data Drift in production NLP?* (By comparing the token frequency distribution of live production data against the training dataset using statistical tests like Population Stability Index (PSI) or Wasserstein Distance).
- *Why do vocabulary differences between training and serving environments crash tokenizers?* (If the serving environment maps token indices using a different dictionary than the training environment, the token indices will map to incorrect word vectors, leading to model degradation).

Module 10: NLP Fundamentals High-Frequency Interview Question Bank

This module provides **40 high-frequency interview questions** covering NLP foundations, mathematical derivations, debugging procedures, and production systems design.

---

1. Conceptual Questions (1-10)

Q1: Contrast workflows vs. classical NLP models vs. modern LLMs across cost, context windows, and latency.

- **Classical NLP Models**: Train specialized parameters (e.g., Bi-LSTM classification head) over static vocabularies. Very low compute cost, sub-10ms inference latency, but limited to short input constraints.
- **Modern LLMs**: Leverage massive attention blocks. High cost and latency, but support long context windows and complex contextual reasoning.
- **Workflows**: Multi-model routing layers connecting classical models (e.g. classifier filtering queries) and LLMs to optimize latency and costs.

Q2: Why did subword tokenization replace character-level and word-level tokenization in modern LLMs?

Word-level tokenization causes a vocabulary size explosion ($|V| > 1,000,000$), increasing memory footprint and resulting in high Out-of-Vocabulary (OOV) rates. Character-level tokenizers solve OOV but increase sequence length, raising attention computational cost $O(L^2)$ quadratically. Subword tokenization (BPE/WordPiece) balances these limits by representing common roots while decomposing unknown words.

Q3: Explain how Byte-Pair Encoding (BPE) handles unseen text sequences during inference.

During BPE training, rare characters are grouped or mapped to individual byte tokens. Unseen words are decomposed into character segments or bytes already present in the vocabulary, preventing `<unk>` mapping errors.

Q4: Explain the differences between the merging objectives of BPE and WordPiece.

BPE merges adjacent token pairs based on raw co-occurrence frequency counts. WordPiece merges pairs that maximize corpus likelihood under a unigram language model, scoring pairs by:

$$\text{Score}(A, B) = \frac{\text{Count}(A, B)}{\text{Count}(A) \times \text{Count}(B)}$$

Q5: How does Unigram tokenization differ from BPE in its building strategy?

BPE is a bottom-up algorithm, starting with individual characters and iteratively merging frequent pairs. Unigram is a top-down algorithm, starting with a large vocabulary and iteratively pruning tokens that have the lowest contribution

to corpus likelihood.

Q6: What is the Distributional Hypothesis and how does it relate to dense word embeddings?

The Distributional Hypothesis states that words occurring in similar contexts share semantic meaning. Dense embeddings learn representations by optimizing models to predict a word given its context (or vice versa), mapping co-occurrence patterns to vector spaces.

Q7: Contrast Continuous Bag of Words (CBOW) and Skip-gram architectures in Word2Vec.

CBOW predicts a single target word given its surrounding context words, averaging context vectors. Skip-gram predicts multiple context words given a target word. Skip-gram performs better on small datasets and rare words, while CBOW trains faster on large corpora.

Q8: How does FastText solve the Out-of-Vocabulary (OOV) problem that causes Word2Vec and GloVe to fail?

FastText represents words as bags of character n-grams. When encountering an OOV token, FastText sums the vectors of its constituent character n-grams to generate an embedding, whereas Word2Vec and GloVe return a generic `<unk>` vector.

Q9: Why do standard Recurrent Neural Networks (RNNs) struggle to model long-range dependencies?

As context length increases, gradients propagated through time are scaled by the recurrent weight matrix product $\prod W_{hh}$. If the spectral radius of W_{hh} is less than 1, the gradient decays exponentially to zero, blinding the model to early tokens.

Q10: Why did Self-Attention replace Recurrent architectures as the standard sequence modeling paradigm?

Recurrent models process tokens sequentially, creating an execution bottleneck. Self-attention models process all tokens in parallel, reducing training path lengths between distant tokens to $O(1)$.

---

2. Mathematical Questions (11-20)

Q11: Derive the TF-IDF calculation for a word that appears twice in a document of 10 words, where the word occurs in 5 out of 100 total documents.

- Term Frequency: $\text{TF} = 2/10 = 0.2$
- Smooth IDF: $\text{IDF} = \log\left(\frac{1+100}{1+5}\right) + 1 = \log(101/6) + 1 \approx \log(16.83) + 1 \approx 2.823 + 1 = 3.823$
- Unnormalized weight: $\text{TF-IDF} = 0.2 \times 3.823 = 0.7646$

Q12: Prove mathematically why scaled dot-product attention scales scores by $1/\sqrt{d_k}$.

Assume components of query $\mathbf{q}$ and key $\mathbf{k}$ are independent random variables with mean 0 and variance 1. The dot product is:

$$u = \sum_{i=1}^{d_k} q_i k_i$$

- Mean: $\mathbb{E}[u] = \sum \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$
- Variance: $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - 0 = (1)(1) = 1$
- Total Variance: $\text{Var}(u) = \sum_{i=1}^{d_k} 1 = d_k$

If d_k is large, the dot product values grow, pushing Softmax into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ scales the input variance to 1:

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$$

Q13: Write the cell state update equation of an LSTM and explain why it prevents vanishing gradients.

- Cell State: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
- Gradient Flow: The derivative with respect to the previous cell state is:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t + \dots$$

> If the forget gate $f_t \approx 1$, the error gradient propagates back through time linearly via addition, bypassing the multiplicative decay of standard RNNs.

Q14: Calculate the Laplace-smoothed bigram probability $P(\text{"cat"} \mid \text{"the"})$ given: $C(\text{"the"}, \text{"cat"}) = 0$, $C(\text{"the"}) = 100$, and vocabulary size $|V| = 10,000$.

$$P_{\text{Laplace}}(\text{"cat"} \mid \text{"the"}) = \frac{C(\text{"the"}, \text{"cat"}) + 1}{C(\text{"the"}) + |V|} = \frac{0 + 1}{100 + 10000} = \frac{1}{10100} \approx 9.9 \times 10^{-5}$$

Q15: Prove that Perplexity is equal to $e^{H(W)}$, where $H(W)$ is the cross-entropy loss of the sequence.

$$\text{PPL}(W) = P(w_1, \dots, w_m)^{-\frac{1}{m}}$$

Taking the natural logarithm:

$$\ln(\text{PPL}(W)) = -\frac{1}{m} \ln P(w_1, \dots, w_m) = -\frac{1}{m} \sum_{i=1}^{m} \ln P(w_i \mid w_{1..i-1}) = H(W)$$

Exponentiating both sides:

$$\text{PPL}(W) = e^{H(W)}$$

Q16: Show how Katz Backoff calculates bigram probabilities when co-occurrence counts are zero.

If $C(w_{i-1}, w_i) = 0$, Katz Backoff estimates the probability by backing off to the lower-order unigram probability, scaled by a normalization factor α:

$$P_{\text{Katz}}(w_i \mid w_{i-1}) = \alpha(w_{i-1}) P(w_i)$$

Q17: Write the loss function for Skip-gram with Negative Sampling (SGNS) and define its components.

$$\mathcal{L} = -\log \sigma(\mathbf{v}'_{w_O} \cdot \mathbf{v}_{w_I}) - \sum_{i=1}^{K} \log \sigma(-\mathbf{v}'_{w_i} \cdot \mathbf{v}_{w_I})$$

- w_I: Input target word.
- w_O: True output context word.
- w_i: Negative sample words.
- K: Number of negative samples.

Q18: Calculate the BLEU brevity penalty (BP) for a generated candidate of 8 words compared to a reference translation of 10 words.

Since candidate length $c = 8 \leq r = 10$:

$$\text{BP} = e^{1-r/c} = e^{1-10/8} = e^{-0.25} \approx 0.7788$$

Q19: Express the GloVe weighting function $f(X_{i,j})$ mathematically and explain its purpose.

$$f(x) = \min\left(1, \left(\frac{x}{x_{\max}}\right)^{\alpha}\right)$$

It bounds the loss contribution of extremely frequent word co-occurrences (e.g. `"the"`, `"and"`), preventing them from dominating the optimization gradient.

Q20: Show the matrix multiplication steps to compute Self-Attention scores for input matrix X.

1. Project inputs: $Q = XW_Q, K = XW_K, V = XW_V$
2. Score similarity: $S = QK^T$
3. Normalize: $A = \text{Softmax}\left(\frac{S}{\sqrt{d_k}}\right)$
4. Output: $Z = AV$

---

3. Production Questions (21-30)

Q21: What is the difference between Data Drift and Concept Drift in production NLP pipelines?

- **Data Drift**: The input text distribution changes (e.g. new vocabulary, slang), but the target relationship remains the same:

$$P(X_{\text{prod}}) \neq P(X_{\text{train}}), \quad P(Y \mid X_{\text{prod}}) = P(Y \mid X_{\text{train}})$$

- **Concept Drift**: The mapping relationship between inputs and outputs shifts (e.g. the word `"viral"` shifts from a negative health context to a positive marketing context):

$$P(Y \mid X_{\text{prod}}) \neq P(Y \mid X_{\text{train}})$$

Q22: How do you identify data drift in a production NLP application?

By measuring statistical divergence between token frequency distributions in production and training data using metrics like Wasserstein Distance or Population Stability Index (PSI).

Q23: Explain the trade-offs of post-training int8 quantization for sequence classifiers.

- **Advantages**: Reduces memory footprint by up to 75%, increases throughput, and lowers VRAM requirements.
- **Disadvantages**: Introduces precision loss, which can degrade classification performance on edge cases.

Q24: How does Feature Hashing save memory in high-vocabulary production models?

It maps words to a fixed-size array index using a hash function, eliminating the need to store a large vocabulary mapping dictionary in memory.

Q25: Why is vocabulary synchronization between training and serving tokenizers critical?

If serving tokenizers use a different dictionary mapping than the training pipeline, token index values will map to incorrect word vectors, degrading model predictions.

Q26: Under what conditions is Cosine Similarity identical to Dot Product?

When the input vectors are pre-normalized to have unit L_2 length ($\|\mathbf{a}\|_2 = \|\mathbf{b}\|_2 = 1$).

Q27: How does BM25 handle term frequency saturation compared to standard TF-IDF?

TF-IDF scales linearly with term frequency, meaning a document repeating a query term 100 times is scored 100 times higher. BM25 scales term frequency non-linearly using a saturation parameter k_1, bounding the maximum score contribution of any single term.

Q28: Detail the step-by-step production debugging feedback loop for resolving model decay.

`Inference` (log predictions) $\rightarrow$ `Metrics Log` (detect accuracy drop or drift) $\rightarrow$ `Error Analysis` (inspect low-confidence predictions) $\rightarrow$ `Diagnostic Actions` (retrain model, update vocabulary) $\rightarrow$ `Re-deployment`.

Q29: What are the latency impacts of choosing Lemmatization over Stemming?

Lemmatization requires part-of-speech (POS) tagging and dictionary lookups, increasing computational latency. Stemming uses rule-based string slicing, which runs significantly faster.

Q30: What is the main trade-off of using greedy decoding vs. beam search decoding?

Greedy search is computationally fast ($O(L)$) but can get stuck in sub-optimal local paths. Beam search explores multiple paths ($O(L \cdot B)$), improving generation quality at the cost of higher latency and memory usage.

---

4. Debugging Questions (31-35)

Q31: How do you diagnose and fix exploding gradients in recurrent sequence models?

- **Diagnosis**: Training loss spikes to `NaN`, or weight gradients show extremely large norms during backpropagation.
- **Fix**: Apply **Gradient Clipping**, scaling down gradients that exceed a threshold:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{\|\mathbf{g}\|}$$

Q32: A sequence classifier returns low accuracy on long sequences. How do you troubleshoot this?

Verify if the model is a standard RNN (suffering from vanishing gradients). If so, replace it with an LSTM or GRU to enable stable gradient flow, or add an attention layer to bypass the sequential bottleneck.

Q33: How do you debug tokenization mismatch bugs where the model output maps to garbage characters?

Check the preprocessing pipeline to ensure training and serving code use the same Unicode normalization (e.g. NFC composition) and subword dictionary indices.

Q34: What causes model performance to degrade when evaluating text containing emojis and URLs?

Standard tokenizers may lack vocabulary tokens for emojis, mapping them to `<unk>`. Fix this by applying pre-tokenization regex cleanups or using a byte-fallback tokenizer.

Q35: Your model's BLEU score is high but human reviewers report poor translation quality. Why?

BLEU only measures exact n-gram overlaps. The model may be generating translations that share n-grams with the reference but contain grammatical errors or inverted meanings (e.g. negations). Resolve this by evaluating with BERTScore or human feedback loops.

---

5. System Design Questions (36-40)

Q36: Design a search query auto-completion system.

- **Ingestion**: Clean input queries, normalize Unicode characters (NFC), and tokenize using BPE.
- **Representation**: Generate n-gram language models from query logs.
- **Retrieval**: Use a trie structure populated with query probabilities. For the current query prefix, look up the top K completions with the highest MLE probabilities.
- **Resilience**: Implement Katz backoff or interpolation to predict completions for unseen or rare prefixes.

Q37: Design a high-throughput spam message classifier.

- **Preprocessing**: Apply regex cleaning to normalize URLs, emojis, and phone numbers.
- **Text Representation**: Use Feature Hashing (to avoid storing a large vocabulary dictionary) to construct sparse term frequency vectors.
- **Classifier**: Train a Naïve Bayes or logistic regression model on hash indices.
- **Production Loop**: Quantize weights to int8, monitor for data drift using Wasserstein distance, and automatically route low-confidence samples to a manual review queue.

Q38: Design an evaluation pipeline for a document summarization service.

- **Metrics**: Compute ROUGE-L (LCS mapping) and BLEU to verify token overlap.
- **Semantic Check**: Generate sentence embeddings using SBERT and measure cosine similarity against reference summaries, and calculate BERTScore to capture contextual token matches.
- **Audit**: Log outputs to an evaluation dataset, routing a percentage of generations to human reviewers for alignment checks.

Q39: Design a Named Entity Recognition (NER) pipeline for streaming financial documents.

- **Preprocessing**: Segment text into sentences, preserving casing (capitalization is a strong signal for financial entities).
- **Tokenization**: Apply WordPiece tokenization.
- **Representation**: Generate contextual token representations using a bidirectional LSTM.
- **Classifier**: Append a Conditional Random Field (CRF) layer to decode the most likely tag sequence.
- **Production**: Deploy the pipeline using real-time streaming inference (e.g. Kafka stream processor).

Q40: Design a semantic search system for an e-commerce platform.

- **Ingestion**: Normalize and tokenize queries using a subword tokenizer (FastText character n-grams to handle OOV typos).
- **Representation**: Retrieve dense query vectors from the embedding layer.
- **Search Index**: Query a vector database (e.g., Faiss) using Hierarchical Navigable Small World (HNSW) indexing to find products with the highest cosine similarity.
- **Fallback**: Fall back to BM25 keyword matching if search vector confidence is low.